Formal Review Request Template

Requester: _Hiram Zaldana_ **Date:** 11/02/26_____
Reviewer: _Jack DiMento_ **Approx. review duration:** 1.5hr_____

Choose **one primary review method**:

Code / Software Review

Code Quality Review
Inspect the code for issues such as readability, modularity, maintainability, efficiency, and appropriate programming practices.
White-Box / Edge-Case Testing
Inspect the internal logic and systematically test branches, boundary conditions, exceptions, or adversarial inputs.
Black-Box / Functional Testing
Develop test cases from the expected requirements or behavior and determine whether the software produces the expected results.
Performance / Runtime Testing
Systematically evaluate whether the software meets relevant runtime or performance expectations.

Usability Review

- **Task-Based Usability Testing**
  Give the reviewer/user specific tasks to complete while you observe what they do, where they succeed, and where they encounter problems. A think-aloud protocol may be used.
- **Heuristic / Guidelines Evaluation**
  Systematically inspect the interface using identified usability principles, heuristics, or design guidelines.
- **Prototype Usability Testing**
  Have users complete defined tasks using a wireframe, prototype, or partially implemented interface.

Design Review

- **Architecture / Conceptual Design Review**
  Evaluate whether architecture and major design decisions align with project requirements and constraints.
- **Preliminary Design / UML Review**
  Evaluate proposed system structure, diagrams, component relationships, interfaces, or other design artifacts.
- **Security Design Review**
  Evaluate relevant authentication flows, trust boundaries, data protection, error handling, or other security-related design decisions.

Other

- **Other review method:**
  _____
  _____

Explain what the reviewer will evaluate and why this method is appropriate for your work:

The reviewer will evaluate two core features of TAG'em Up: CSV data importing and GitHub integration. Black-box functional testing is appropriate because both features involve processing external data and displaying results within the application. The review will help determine

whether these features work as expected, handle common errors appropriately, and provide accurate feedback to users.

Review Request Details

1) What specifically should the reviewer examine or interact with?

Uploading a student csv file containing student info and tracking Teams github activity.

CSV Import

- Test importing a properly formatted CSV file containing valid data.
- Verify that the imported records appear correctly within the application.
- Test an incorrectly formatted CSV file or one containing missing or invalid values.
- Observe how the application handles invalid inputs, duplicate records, and import errors.

GitHub Integration

- Navigate to the Team Status Page and examine the GitHub-related metrics.
- Verify that the application retrieves information from the team's configured GitHub repository and project board.
- Compare displayed task counts, assignments, time estimates, and completed time against the GitHub Project board.
- Examine how the application handles missing or unavailable GitHub data.

---

2) What question do you want this review to help answer?

Examples:

- Does this implementation correctly handle boundary and error cases?
- Can a new user complete the checkout task without assistance?
- Does this architecture satisfy our project's stated requirements?

Do the CSV Import and GitHub Integration features correctly retrieve, process, and display data while providing appropriate feedback when errors or unexpected inputs occur?

---

3) What should the reviewer focus on?

- **Functional correctness:** Whether both features produce the expected results.

- **Data accuracy:** Whether imported records and GitHub metrics match their original sources.
- **Error handling:** Whether invalid inputs or unavailable data are handled appropriately.
- **Usability:** Whether the features are understandable and provide clear feedback during normal use.

For GitHub integration, particular attention should be given to the accuracy of Kanban task counts, individual assignments, time estimates, and progress calculations.

---

4) What should the reviewer *not* evaluate during this review?

Examples:

- Do not evaluate visual polish or mobile responsiveness (for a usability test evaluating navigation and task completion)
- Do not evaluate the database layer or unfinished API integration (for a code review focusing on the authentication module)

The reviewer does not need to evaluate the application's overall visual design, mobile responsiveness, or unrelated features.

A comprehensive code-quality review, database architecture review, and performance benchmarking are also outside the scope of this request.

The GitHub integration is still under development, particularly its support for tracking contributions across feature branches and its handling of sprint-specific task attribution. These limitations should be noted but are not the primary focus of this review.

---

5) What should the reviewer use to judge the work?

Examples include expected software behavior, project requirements, coding best practices, usability heuristics, or design constraints.

The reviewer should evaluate the features using their expected functionality and the accuracy of their outputs.

For CSV Import, successful behavior means valid records are imported correctly, while invalid data is rejected or handled appropriately without unintended changes.

For GitHub Integration, successful behavior means the application retrieves available project data and displays metrics consistent with the corresponding GitHub repository and project board.

The reviewer should document unexpected behavior, incorrect calculations, unclear feedback, or errors encountered during testing.

Reviewer Section

1) Briefly describe what you actually did during the review.

   I opened the new updated project and made a new semester. Added a team to it and imported their csv. Once that was done, I could add their github token to their team overview. Then I went to the team status and linked the github repo using the token and made sure that data was imported successfully.

Examples include executing defined test cases, inspecting particular code paths, observing a user completing defined tasks, or systematically examining a design against specified criteria.

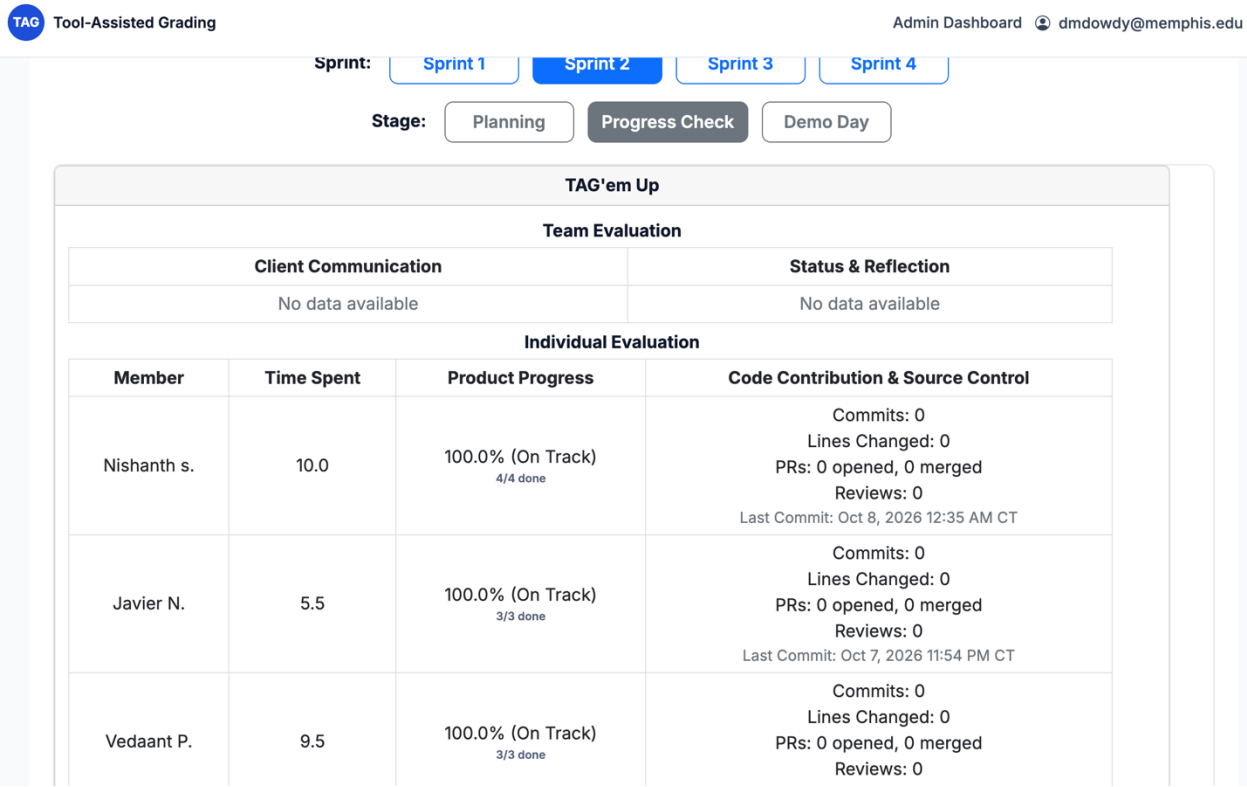
---

2) What portions of the artifact did you evaluate?

   I operated on their working site going through setup, login, semester/semester creation, team overview/editing, and team status.

---

3) Provide or link to relevant evidence such as test cases, screenshots, notes, annotated code, observations, or other review artifacts.



Findings

Strengths:

- I got through the entire flow without running into a single problem.
- The way everything is connected seemed to be very responsive/fast

Issues / Defects:

Description of Issue	Severity (Low/Med/High)	Evidence / Example	Suggestions
Slight trouble finding where to implement github token	low	Had a little trouble finding where to input github token. (I was looking in semester tab and not team overview) I think you guys flow works but confused me a little.	Maybe if you could add the github token when you are making the semester/tea m that would make the flow a little easier. Than making everything and then going back to edit it.
Couldn't make a 2026 semester like requested	low	There was already a 2026 semester in there, and flow stopped me from creating a duplicate so I made a 2027 semester	I think this should be kept as is I am only listing it as an issue because I found nothing else and was told to make a 2026 semester

Requester Follow-Up

1) How will you address the feedback? If you decide **not** to, briefly explain why.